



Dr. D. Y. PATIL VIDYAPEETH, PUNE
(Deemed to be University)

DR. D. Y. PATIL SCHOOL OF SCIENCE AND TECHNOLOGY

TATHAWADE, PUNE

A Mini- Project Report on

...Space Shooter...

SUBMITTED BY:

NAME OF STUDENT

ROLL NUMBER

1. Harshal Malviya

BTAI-13

2. Krish Kumar

BTAI-24

3.Suridhi Gupta

BTAI-54

4.Suyash Raj Singh

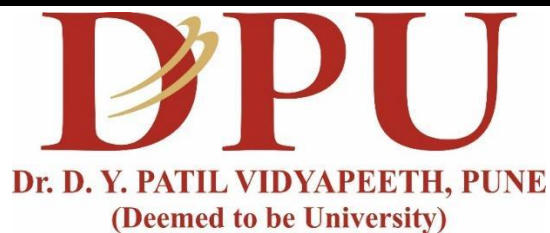
BTAI-55

GUIDED BY:

Mrs Khushbu Wase

ARTIFICIAL INTELLIGENCE & DATA SCIENCE

ACADEMIC YEAR 2025-2026



DR. D. Y. PATIL SCHOOL OF SCIENCE AND TECHNOLOGY

TATHAWADE, PUNE

CERTIFICATE

This is to certify that the Mini- Project Report entitled

-“ Space Shooter “-

is a bonafide work carried out by Mr Suyash Raj under the supervision of **Mrs Khushbu Wase** and it is submitted towards the partial fulfillment of the requirement Project of Web Technology.

Mrs Khushbu Wase

Project Guide

Dr. Mily Lal

HOD (DYPSST)

ARTIFICIAL INTELLIGENCE & DATASCIENCE

ACADEMIC YEAR 2025-2026

ABSTRACT

This project is a fully web-based Space Shooter Game developed using HTML, CSS, and JavaScript. The aim of the project is to create an interactive and visually appealing game that demonstrates the practical application of front-end web development concepts, real-time animation, and event-driven programming. The game consists of multiple pages including a homepage, instructions page, customization page, and the main game screen. HTML is used to structure all these pages, while CSS provides styling, layout, colors, animations, and overall interface design to enhance the user experience.

The core functionality of the game is implemented using JavaScript. The player controls a spaceship that can move horizontally and shoot bullets to destroy enemies descending from the top. Key features such as enemy movement, collision detection, score updates, and game-over conditions are implemented using continuous game loops. Additional JavaScript modules like `backgroundStars.js` and `spaceBackground.js` generate animated backgrounds to make the gameplay visually dynamic. The `music.js` file handles sound effects and background music, making the gameplay immersive. These modular files help maintain clean and organized code.

The project also includes a Customization Page, allowing players to choose different spaceship designs, adding a personal touch to the game. Real-time event handling is used to capture keyboard inputs for movement and shooting. CSS transitions and animations enhance the smoothness of the visuals, while JavaScript ensures accurate updates per frame, giving the game a polished feel.

Through the development of this game, we gained hands-on experience with DOM manipulation, canvas animations, timing functions, asset handling, modular coding, and responsive UI design. Overall, this Space Shooter Game successfully demonstrates how modern web technologies can be used to build interactive and engaging applications, providing a strong foundation for creating more advanced browser-based games in the future.

INDEX

SR.NO	TOPIC	PAGE NO
1]	INTRODUCTION	6-7
	1.1 Objectives	6
	1.2 Problem Statements	7
2]	DESIGN	8-9
	2.1 System Architecture	8
	2.2 Use Case Diagram	9
3]	ALGORITHMS & FLOWCHARTS	10-17
4]	IMPLEMENTATION OF CODE	18-28
5]	OUTPUT	29-32
6]	CONCLUSION	33
7]	REFERENCES	34

List of Figures

SR.NO	FIGURE NO.	PAGE NO
1.	Fig.1.1 (System Architecture)	8
2.	Fig.1.2 (Use Case Diagram)	9
3.	Fig.1.3(Flowcharts)	17
4.	Fig1.4.(Home.html)	18
5.	Fig1.5.(Instruction.html)	19
6.	Fig.1.6(Customize.html)	20
7.	Fig.1.7(Game.html)	21
8.	Fig.1.8(Customize.css)	22
9.	Fig.1.9.(Game.css)	23
10.	Fig.1.10(Style.css)	24
11.	Fig.1.11(BackgroundStars.js)	25
12.	Fig.1.12(Game.js)	26
13.	Fig.1.13.(Music.js)	27
14.	Fig.1.14(spaceBackground.js)	28
15.	Fig.1.15(Home Page)	29
16.	Fig1.16(Instruction Page)	29-30
17.	Fig.1.17(Customize Spaceship Page)	30
18.	Fig.1.18(Main Game)	31
19.	Fig.1.19(Powerup[Lives[]])	31
20.	Fig.1.20(Bot)	32
21.	Fig.1.21(End Game)	32

Chapter 1

INTRODUCTION

The Space Shooter Game is a web-based interactive application developed using HTML, CSS, and JavaScript. The project aims to demonstrate how modern front-end technologies can be used to build a fully functional game that runs directly in a web browser without requiring any additional software. The game provides an engaging experience where the player controls a spaceship, shoots enemies, earns points, and navigates through a visually dynamic space environment.

Through this project, we explored real-time animations, keyboard event handling, DOM manipulation, modular scripting, and responsive design. The project also includes multiple web pages such as the homepage, instructions page, customization page, and the main gameplay page, making it a complete mini-website along with a game. The game's design focuses on both entertainment and learning, helping us understand how web technologies can be applied creatively to build interactive applications.

1.1 Objectives

1. To design a simple and interactive web-based game using HTML, CSS, and JavaScript.
2. To implement real-time animations, enemy movement, and collision detection using JavaScript.
3. To create a multi-page structure including homepage, instructions, customization, and game screen.
4. To enhance user experience with background animations, sound effects, and smooth transitions.
5. To apply event-driven programming using keyboard inputs for controlling the spaceship.
6. To practice clean code structure through modular JavaScript files for background, music, and game logic.
7. To understand the practical application of front-end development concepts in game creation.

1.2 Problem Statement

Traditional web development often focuses only on static websites or simple interactive forms. Students rarely get to explore real-time animations, game loops, and dynamic user interactions within a browser environment. There is also a lack of beginner-friendly projects that combine creativity with technical learning.

This project addresses these gaps by developing a browser-based Space Shooter game that incorporates real-time movement, collision detection, multi-page navigation, visual effects, and user-controlled gameplay. It demonstrates how front-end technologies can be used not only for websites but also for building engaging and interactive applications.

Chapter 2

DESIGN

2.1 System Architecture

This diagram ([Fig.1.1](#)) represents the system architecture of the Space Shooter Game, showing how the entire application runs within the user's web browser. The structure starts with the user interacting through the browser, which then loads the HTML pages for different screens, applies CSS for styling and layout, and uses JavaScript files to control all game logic, animations, and interactions. The assets, including images and audio, support the visual and sound elements of the game. Together, these components work in coordination to deliver a smooth, interactive, and fully client-side gaming experience.

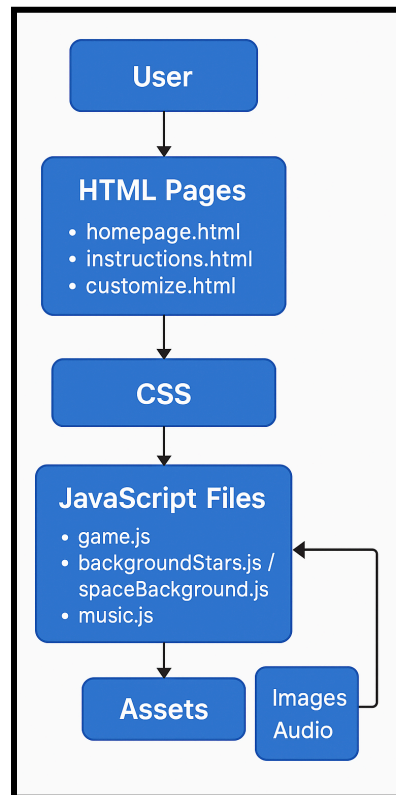


Fig.1.1 (System Architecture)

Overall Flow

User loads webpage → selects options → enters game → JS renders game loop → game ends → user can restart or go back home.

2.2 Use Case Diagram

The Use Case Diagram ([Fig.1.2](#)) illustrates how the player interacts with different features of the Space Shooter Game. It highlights the player as the main actor who can start the game, view instructions, customize the spaceship, and navigate through various options. During gameplay, the player controls movement, shoots enemies, earns points, and eventually reaches a game-over state. The diagram shows all major actions the user can perform and how each part of the system responds, giving a clear overview of the game's functionality from the user's perspective.

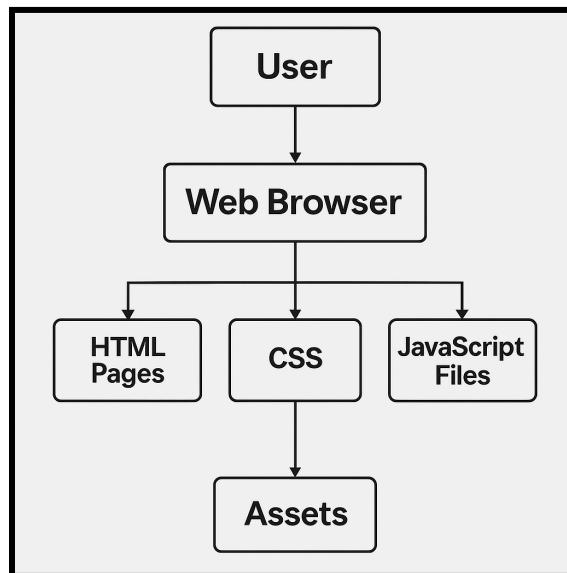


Fig.1.2 (Use Case Diagram)

Chapter 3

ALGORITHMS & FLOWCHARTS

This section explains the key algorithms and flowcharts used in developing the Space Shooter Game. It describes how the game initializes, processes user input, updates objects, and handles collisions. The algorithms ensure smooth gameplay by managing enemy movement, bullet firing, and scoring. Flowcharts provide a visual understanding of the game's internal processes and logic structure. Together, they show how HTML, CSS, and JavaScript work to create a fully functional interactive game.

3.1 ALGORITHMS

3.1.1. Initialize Game :

Purpose: load assets, set initial variables, bind events.

Code:

```
Algorithm InitializeGame()

    Load images (player, enemy, bullet, backgrounds)

    Load audio (bgMusic, shootSound, explosionSound)

    Set canvas width, height

    Set player = {x: centerX, y: bottomY, speed, lives}

    enemies = empty list

    bullets = empty list

    score = 0

    gameState = "menu"    // or "playing", "paused", "gameover"

    lastEnemySpawnTime = currentTime()

    Bind keyboard events: keydown, keyup
```

```
    Start background animations (if any)
    If gameState == "playing" then Start GameLoop()
End Algorithm
```

Notes: keep assets preloaded to avoid runtime stutter.

3.1.2. Main Game Loop

Purpose: update game state each frame and render.

Code:

```
Algorithm GameLoop()
    while gameState == "playing":
        dt = time since last frame
        HandleInput(dt)
        UpdateEntities(dt)
        CheckCollisions()
        RemoveOffscreenEntities()
        RenderFrame()
        If checkGameOverCondition() then gameState = "gameover"
        requestAnimationFrame(GameLoop)
    end while
End Algorithm
```

Complexity: $O(E + B)$ per frame where E=enemies, B=bullets.

3.1.3. Handle Input

Purpose: move player and fire bullets based on keys.

Code:

```
Algorithm HandleInput(dt)
    If leftKey pressed then player.x -= player.speed * dt
    If rightKey pressed then player.x += player.speed * dt
    If shootKey pressed and canShoot():
        Create bullet at (player.x, player.y)
        Play shootSound()
        set bulletCooldown
End Algorithm
```

3.1.4. Spawn Enemies

Purpose: generate enemies periodically with varying properties.

Code:

```
Algorithm SpawnEnemies()
    if currentTime - lastEnemySpawnTime >= enemySpawnInterval:
        enemy = createEnemy(randomX(), startY, speed=randomRange)
        enemies.append(enemy)
        lastEnemySpawnTime = currentTime
        possiblyDecreaseSpawnInterval() // increases difficulty
End Algorithm
```

3.1.5. Update Entities

Purpose: move enemies, bullets, animate backgrounds.

Code:

```
Algorithm UpdateEntities(dt)
    for each bullet in bullets:
        bullet.y -= bullet.speed * dt
    for each enemy in enemies:
        enemy.y += enemy.speed * dt
        if enemy has pattern then update pattern (e.g., zigzag)
    UpdateBackgroundAnimations(dt)
End Algorithm
```

3.1.6. Collision Detection

Purpose: detect bullet-enemy and enemy-player collisions.

Code:

```
Algorithm CheckCollisions()
    for each bullet in bullets:
        for each enemy in enemies:
            if intersects(bullet, enemy):
                remove enemy
                remove bullet
                score += enemy.pointValue
```

```

        play explosionSound()
        break inner loop
    for each enemy in enemies:
        if intersects(enemy, player):
            handlePlayerHit()
End Algorithm

Function intersects(a, b):
    // simple AABB collision
    return (a.x < b.x + b.width) and (a.x + a.width > b.x) and
           (a.y < b.y + b.height) and (a.y + a.height > b.y)
End Function

```

Notes: optimize using spatial partition if enemies get large in number.

3.1.7. Remove Offscreen Entities

Code:

```

Algorithm RemoveOffscreenEntities()
    bullets = bullets.filter(b => b.y > -buffer)
    enemies = enemies.filter(e => e.y < canvas.height + buffer)
End Algorithm

```

3.1.8. Game Over Handling

Code:

```
Algorithm checkGameOverCondition()
    if player.lives <= 0 then return true
    else return false
End Algorithm
```

```
Algorithm handleGameOver()
    Stop game loop (or set gameState="gameover")
    Show game over screen with score and restart option
    Save high score (if implemented)
End Algorithm
```

3.1.9. Customization (choose ship)

Code:

```
Algorithm OpenCustomization()
    Display ship options to user
    On user select:
        player.sprite = selectedSprite
        saveSelectionToLocalStorage()
End Algorithm
```

3.1.10. Music & Sound Control

Code:

```
Algorithm ToggleMusic()  
    if music.playing then music.pause()  
    else music.play()  
    updateUIIcon()  
End Algorithm
```

3.2. Flowcharts

The flowcharts (Fig1.3) visually explain how the Space Shooter Game operates internally. They show the step-by-step logic of the game, starting from initialization to the main game loop and ending at the game-over state. Each diagram illustrates how the system handles user input, spawns enemies, updates movements, detects collisions, and renders each frame on the screen. The flowcharts make it easier to understand the game's structure by converting the algorithms into a visual process, helping readers clearly see how the game runs in a continuous loop until the player loses. Overall, they provide a simple, graphical representation of how the core functions of the game interact with each other.

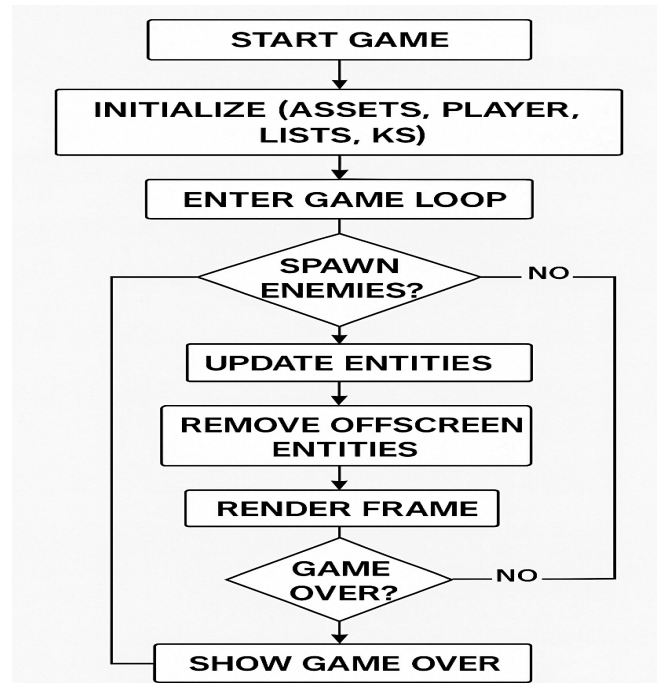


Fig.1.3(Flowcharts)

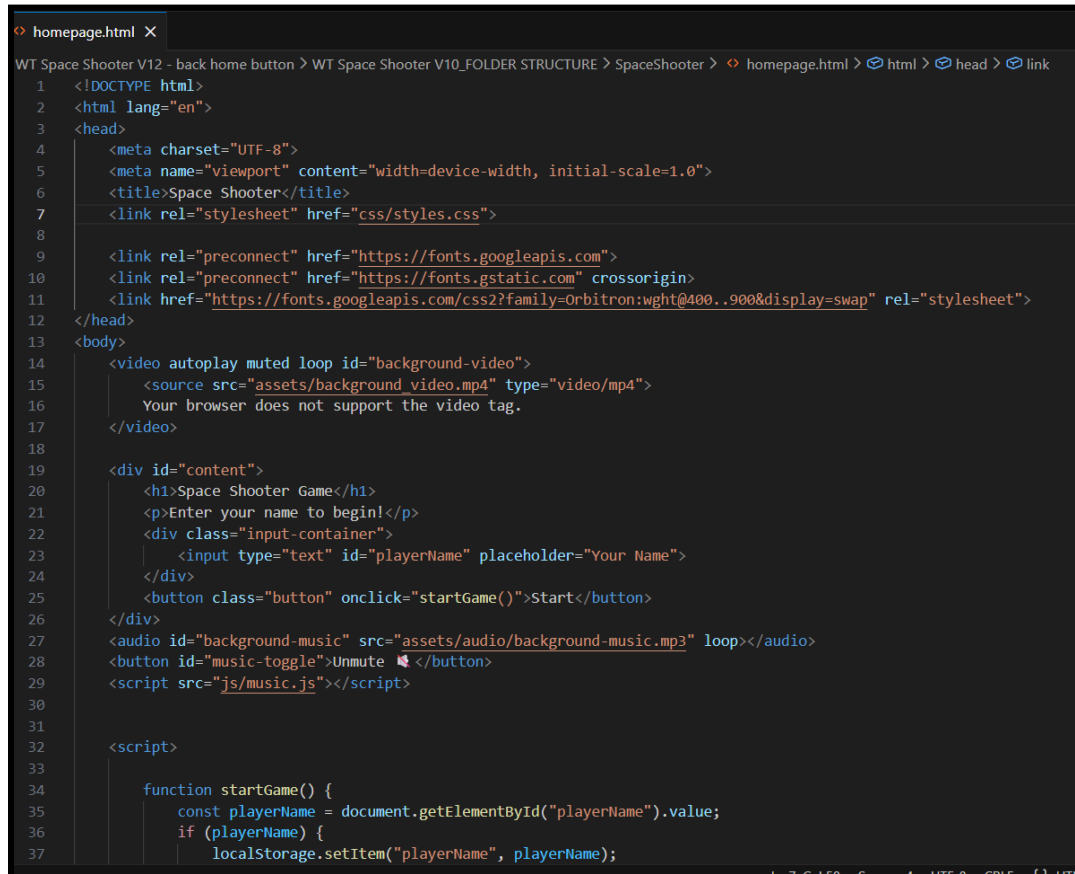
Extra details :

- **Collision method:** AABB (axis-aligned bounding box) for simplicity. Mention pixel-perfect optional improvement.
- **Difficulty scaling:** reduce `enemySpawnInterval` or increase `enemy.speed` every N points.
- **Optimization note:** If enemies > 100, consider quad-tree or grid-based partitioning to reduce collision checks from $O(E \cdot B)$.
- **Storage:** `localStorage` used to save selected ship and high score (if implemented).

Chapter 4

IMPLEMENTATION OF CODE

4.1.Home.html :

A screenshot of a code editor with a dark theme. The editor shows the HTML code for a file named 'homepage.html'. The code includes a DOCTYPE declaration, HTML and head tags, meta tags for charset and viewport, a title 'Space Shooter', and links to a stylesheet and Google Fonts. The body contains a video player, a content div with a title, a paragraph, an input field, and a 'Start' button. It also includes an audio player, an 'Unmute' button, and a JavaScript function 'startGame()' that saves the player's name to local storage.

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Space Shooter</title>
7   <link rel="stylesheet" href="css/styles.css">
8
9   <link rel="preconnect" href="https://fonts.googleapis.com">
10  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
11  <link href="https://fonts.googleapis.com/css2?family=Orbitron:wght@400.900&display=swap" rel="stylesheet">
12 </head>
13 <body>
14   <video autoplay muted loop id="background-video">
15     <source src="assets/background video.mp4" type="video/mp4">
16     Your browser does not support the video tag.
17   </video>
18
19   <div id="content">
20     <h1>Space Shooter Game</h1>
21     <p>Enter your name to begin!</p>
22     <div class="input-container">
23       <input type="text" id="playerName" placeholder="Your Name">
24     </div>
25     <button class="button" onclick="startGame()">Start</button>
26   </div>
27   <audio id="background-music" src="assets/audio/background-music.mp3" loop></audio>
28   <button id="music-toggle">Unmute </button>
29   <script src="js/music.js"></script>
30
31
32   <script>
33     function startGame() {
34       const playerName = document.getElementById("playerName").value;
35       if (playerName) {
36         localStorage.setItem("playerName", playerName);
37       }
```

Fig1.4.(Home.html)

4.2.Instruction.html :

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Mission Briefing</title>
7   <link rel="stylesheet" href="css/styles.css">
8   <link rel="preconnect" href="https://fonts.googleapis.com">
9   <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
10  <link href="https://fonts.googleapis.com/css2?family=Orbitron:wght@400..900&display=swap" rel="stylesheet">
11 </head>
12 <body class="page-instructions">
13   <audio id="background-music" src="assets/audio/background-music.mp3" loop></audio>
14   <button id="music-toggle">Unmute </button>
15
16   <video autoplay muted loop id="background-video">
17     <source src="assets/background_video.mp4" type="video/mp4">
18     Your browser does not support the video tag.
19   </video>
20
21   <div id="content">
22     <h1 class="instructions-title">Mission Briefing</h1>
23     <p id="playerNameGreeting" class="greeting"></p>
24
25     <div class="instructions-list">
26       <div class="controls-grid">
27         <section class="instruction-section">
28           <h2><span class="icon"></span> Pilot Controls</h2>
29           <p>Use the <strong>ARROW KEYS</strong> to navigate the void and evade enemy fire.</p>
30         </section>
31         <section class="instruction-section">
32           <h2><span class="icon"></span> Weapon Systems</h2>
33           <p>Press the <strong>SPACEBAR</strong> to unleash your primary plasma cannons.</p>
34         </section>
35         <section class="instruction-section">
36           <h2><span class="icon"></span> Tactical Drone</h2>
37           <p>Press the <strong>'P' KEY</strong> to deploy or recall your autonomous combat drone.</p>
38         </section>
39       </div>
40     </div>
41   </div>
42 </body>
43 </html>
```

Fig1.5.(Instruction.html)

4.3.Customize.html :

```
customize.html X
WT Space Shooter V12 - back home button > WT Space Shooter V10_FOLDER STRUCTURE > SpaceShooter > customize.html > ...
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Customize Your Ship</title>
7   <link rel="stylesheet" href="css/customize.css">
8   <link rel="preconnect" href="https://fonts.googleapis.com">
9   <link rel="preconnect" href="https://fonts.gstatic" crossorigin>
10  <link href="https://fonts.googleapis.com/css2?family=Orbitron:wght@400.900&display=swap" rel="stylesheet">
11 </head>
12 <body>
13   <audio id="background-music" src="assets/audio/background-music.mp3" loop></audio>
14   <button id="music-toggle">Unmute 🔊 </button>
15   <script src="js/music.js"></script>
16
17   <canvas id="starBackground"></canvas>
18
19   <div class="container">
20     <h1>Choose Your Ship</h1>
21     <div class="ship-options">
22       <div class="ship-card" data-ship="red">
23         <div class="ship-preview red"></div>
24         <p>Red Falcon</p>
25       </div>
26       <div class="ship-card active" data-ship="blue">
27         <div class="ship-preview blue"></div>
28         <p>Blue Comet</p>
29       </div>
30       <div class="ship-card" data-ship="green">
31         <div class="ship-preview green"></div>
32         <p>Green Stinger</p>
33       </div>
34     </div>
35     <button id="startGameBtn">Start Game</button>
36   </div>
37
```

Fig.1.6(Customize.html)

4.4.Game.html :

```
game.html X
WT Space Shooter V12 - back home button > WT Space Shooter V10_FOLDER STRUCTURE > SpaceShooter > game.html > ...
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4    <meta charset="UTF-8">
5    <meta name="viewport" content="width=device-width, initial-scale=1.0">
6    <title>Space Shooter Game</title>
7    <link rel="stylesheet" href="css/game.css">
8    <link rel="preconnect" href="https://fonts.googleapis.com">
9    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
10   <link href="https://fonts.googleapis.com/css2?family=Orbitron:wght@400..900&display=swap" rel="stylesheet">
11 </head>
12 <body>
13   <canvas id="starBackground"></canvas>
14
15   <div id="gameContainer">
16     <div id="explosionEffect"></div>
17     <canvas id="gameCanvas"></canvas>
18
19     <div id="scoreBoard">
20       <span id="playerNameDisplay"></span>
21       <span id="score">Score: 0</span>
22       <span id="highScore">High Score: 0</span>
23       <span id="lives">Lives: 3</span>
24     </div>
25
26     <div id="gameOver">
27       <h1>GAME OVER</h1>
28       <p>Your Score: <span id="finalScore">0</span></p>
29       <p id="finalHighScoreMessage"></p>
30
31       <div class="button-container">
32         <button id="restart-btn" onclick="restartGame()">Restart</button>
33         <a href="homepage.html">
34           <button class="back-home-btn">Back Home</button>
35       </a>
```

Fig.1.7(Game.html)

4.5. Customize.css :

```
# customize.css X
WT Space Shooter V12 - back home button > WT Space Shooter V10_FOLDER STRUC
1  body {
2      background: black;
3      color: white;
4      font-family: 'Orbitron', sans-serif; /* Using the co
5      display: flex;
6      justify-content: center;
7      align-items: center;
8      height: 100vh;
9      margin: 0;
10     overflow: hidden; /* Hide scrollbars */
11 }
12
13 #starBackground {
14     position: fixed;
15     top: 0;
16     left: 0;
17     width: 100vw;
18     height: 100vh;
19     background-color: black;
20     z-index: -1; /* Place it behind the content */
21     opacity: 1;
22 }
23
24 .container {
25     text-align: center;
26     z-index: 1; /* Ensure content is above the canvas */
27 }
28
29 h1 {
30     font-size: 3em;
31     margin-bottom: 50px;
32     color: cyan;
33     text-shadow: 0 0 10px cyan;
34 }
35
36 .ship-options {
37     display: flex;
```

Fig.1.8(Customize.css)

4.6.Game.css :

```
# game.css X
WT Space Shooter V12 - back home button > WT Space Shooter V10_FOLDER STRUCTURE > SpaceSho

1  * {
2    margin: 0;
3    padding: 0;
4    box-sizing: border-box;
5  }
6
7  body {
8    background: radial-gradient(circle at 20% 20%, #050505, #000);
9    color: white;
10   font-family: 'Orbitron', sans-serif;
11   text-align: center;
12   overflow: hidden;
13   height: 100vh;
14 }
15
16 #gameContainer {
17   position: relative;
18   width: 30vw;
19   margin: 0 auto;
20   height: 100vh;
21   overflow: hidden;
22 }
23
24 /* ----- Canvas ----- */
25 #gameCanvas {
26   background: transparent;
27   display: block;
28   width: 100%;
29   height: 100%;
30 }
31
32 #scoreBoard {
33   position: absolute;
34   top: 10px;
35   left: 50%;
36   transform: translateX(-50%);
37   font-size: 18px;
```

Fig.1.9.(Game.css)

4.7.Style.css :

```
# styles.css X
WT Space Shooter V12 - back home button > WT Space Shooter V10_FOLDER STRUCTURE > SpaceShooter
1  /* styles.css */
2
3  body {
4      background-color: black;
5      color: white;
6      font-family: 'Orbitron', sans-serif; /* USE THE NEW FONT */
7      text-align: center;
8      margin: 0;
9      overflow: hidden; /* Prevent scrollbars from the video */
10 }
11
12 /* New video background style */
13 #background-video {
14     position: fixed;
15     top: 0;
16     left: 0;
17     width: 100%;
18     height: 100%;
19     object-fit: cover; /* This makes the video cover the screen without d
20     z-index: -1; /* This is key to placing it behind other content */
21 }
22
23 /* Wrap your content to layer it above the canvas */
24 #content {
25     display: flex;
26     flex-direction: column;
27     justify-content: center;
28     align-items: center;
29     height: 100vh;
30     position: relative; /* Ensures it respects z-index */
31     z-index: 1;
32
33     /* --- ADD/CHANGE THESE STYLES --- */
34     background: rgba(0, 15, 30, 0.4); /* Dark blue semi-transparent par
35     padding: 2rem 3rem;
36     border: 1px solid cyan; /* Cyan border */
37     border-radius: 10px;
```

Fig.1.10(Style.css)

4.8. BackgroundStars.js :

```
1 // backgroundStars.js
2
3 const bgCanvas = document.getElementById('starBackground');
4 const bgCtx = bgCanvas.getContext('2d');
5
6 let bgStars = [];
7 // Use a larger number of stars to fill the full screen adequately.
8 const NUM_BG_STARS = 400;
9
10 /**
11  * Sets the background canvas size to fill the window and recreates the
12  */
13 function resizeBgCanvas() {
14     bgCanvas.width = window.innerWidth;
15     bgCanvas.height = window.innerHeight;
16     createBgStars();
17 }
18
19 /**
20  * Populates the star array with random positions, sizes, and speeds.
21  */
22 function createBgStars() {
23     bgStars = []; // Clear existing stars
24     for (let i = 0; i < NUM_BG_STARS; i++) {
25         bgStars.push({
26             x: Math.random() * bgCanvas.width,
27             y: Math.random() * bgCanvas.height,
28             // Smaller radius for a more subtle, distant look
29             radius: Math.random() * 1.5,
30             // Slower speed to create a parallax effect against the game
31             speed: Math.random() * 2 + 0.5
32         });
33     }
34 }
35
36 /**
37  * The main animation loop for the background.
```

Fig.1.11(BackgroundStars.js)

4.9.Game.js :

```
const canvas = document.getElementById("gameCanvas");
const ctx = canvas.getContext("2d");

// --- IMAGE LOADING ---
const spaceshipImg = new Image();
const droneImg = new Image();
droneImg.src = 'assets/images/drone.png';
const enemyImg = new Image();
enemyImg.src = 'assets/images/enemy.png';

// --- SELECT SHIP BASED ON PLAYER'S CHOICE ---
const selectedShip = localStorage.getItem('selectedShip');
✓ if (selectedShip === 'red') {
    spaceshipImg.src = 'assets/images/red-Falcon.png';
✓ } else if (selectedShip === 'green') {
    spaceshipImg.src = 'assets/images/green-stinger.png';
✓ } else {
    spaceshipImg.src = 'assets/images/spaceship.png';
}

// --- GAME VARIABLES ---
let stars = [];
const NUM_STARS = 200;

let spaceship = { x: canvas.width / 2 - 40, y: canvas.height - 100, width: 80,
let bullets = [];
let enemies = [];
let enemyBullets = [];

// Drone variables using width/height for the image
let droneEnabled = false;
let drone = { x: 0, y: 0, width: 35, height: 35 };
let droneBullets = [];
let lastDroneShot = 0;
let boosted200 = false;
let boosted300 = false;
let boosted500 = false;
```

Fig.1.12(Game.js)

4.10.Music.js :

```
JS music.js X
WT Space Shooter V12 - back home button > WT Space Shooter V10_FOLDER STRUCTURE > SpaceShooter > js > JS
1  window.addEventListener('load', () => {
2      const backgroundMusic = document.getElementById('background-music');
3      const musicToggleButton = document.getElementById('music-toggle');
4
5      // This function handles the logic for playing or pausing the music
6      function toggleMusic() {
7          if (!backgroundMusic) return; // Safety check
8
9          if (backgroundMusic.paused) {
10              backgroundMusic.play();
11              localStorage.setItem('musicState', 'playing');
12              if (musicToggleButton) musicToggleButton.textContent = 'Mute 🔊';
13          } else {
14              backgroundMusic.pause();
15              localStorage.setItem('musicState', 'paused');
16              if (musicToggleButton) musicToggleButton.textContent = 'Unmute 🔊';
17          }
18      }
19
20      // The clickable button will still work
21      if (musicToggleButton) {
22          musicToggleButton.addEventListener('click', toggleMusic);
23      }
24
25      // --- NEW CODE ADDED HERE ---
26      // Add a listener for keyboard presses
27      window.addEventListener('keydown', (event) => {
28          // Check if the key pressed was 'm' (case-insensitive)
29          if (event.key.toLowerCase() === 'm') {
30              toggleMusic();
31          }
32      });
33      // --- END OF NEW CODE ---
34
35      // Logic to continue music across pages
36      const musicState = localStorage.getItem('musicState');
37      const musicTime = localStorage.getItem('musicTime');
```

Fig.1.13.(Music.js)

4.11.spaceBackground.js :

```
JS spaceBackground.js X
WT Space Shooter V12 - back home button > WT Space Shooter V10_FOLDER STRUCTURE > SpaceShooter > js > JS spaceBackgro

1  // spaceBackground.js
2
3  const canvas = document.getElementById('spaceBackground');
4  const ctx = canvas.getContext('2d');
5
6  // --- Entity Arrays ---
7  let stars = [], asteroids = [], nebulas = [], galaxies = [], alienShips = [], meteors = [],
8
9  // --- Constants ---
10 const NUM_STARS = 400;
11 const NUM_ASTEROIDS = 7;
12 const NUM_NEBULAS = 2; // Less is more for realism
13 const NUM_GALAXIES = 2;
14 const NUM_ALIEN_SHIPS = 2; // More subtle and rare
15 const NUM_DUST = 700;
16 const NUM_PLANETS = 1;
17
18 // --- UTILITY FUNCTIONS ---
19 function random(min, max) { return Math.random() * (max - min) + min; }
20
21 // --- OBJECT CREATION ---
22
23 function createStar() {
24   const layer = random(1, 4);
25   return {
26     x: random(0, canvas.width), y: random(0, canvas.height),
27     size: random(0.4, layer === 1 ? 2.0 : layer === 2 ? 1.2 : 0.6),
28     speed: layer === 1 ? 0.35 : layer === 2 ? 0.2 : 0.1,
29     alpha: random(0.5, 1),
30     twinkleSpeed: random(0.005, 0.01), twinkleState: 1,
31     color: Math.random() > 0.9 ? (Math.random() > 0.5 ? '■' : '■') : '■' :
32   };
33 }
34
35 function createAsteroid() {
36   const size = random(8, 35);
37   return {
```

Fig.1.14(spaceBackground.js)

Chapter 5

OUTPUT

5.1.Home Page Appearance :

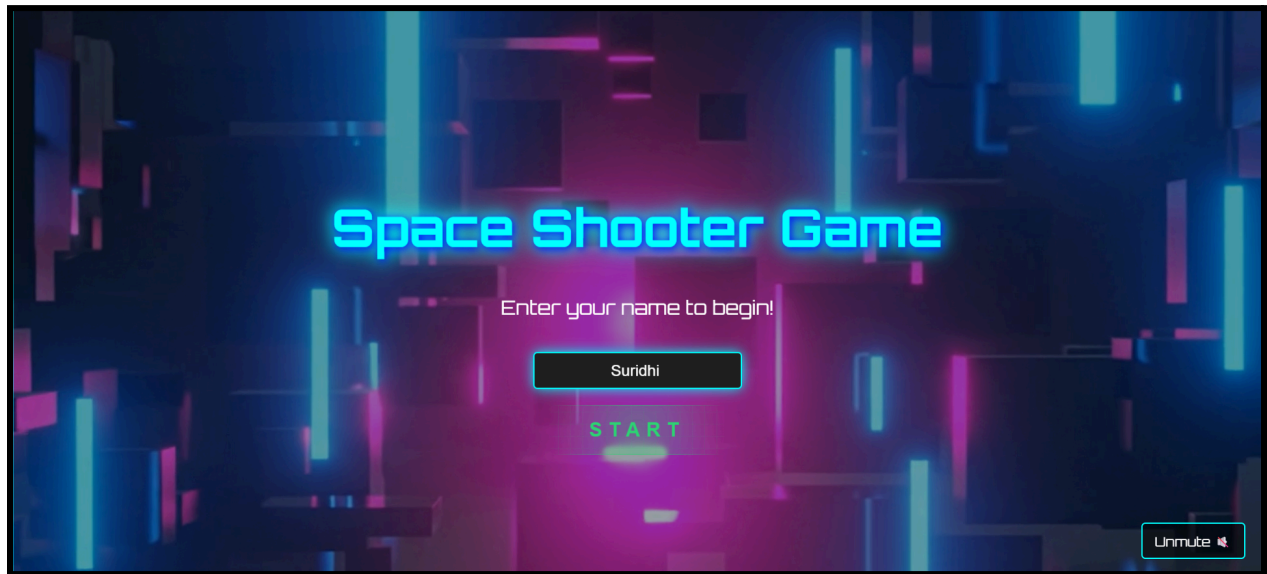


Fig.1.15(Home Page)

5.1.Instructions Page :





Fig1.16(Instruction Page)

5.3.Customize Spaceship Page :

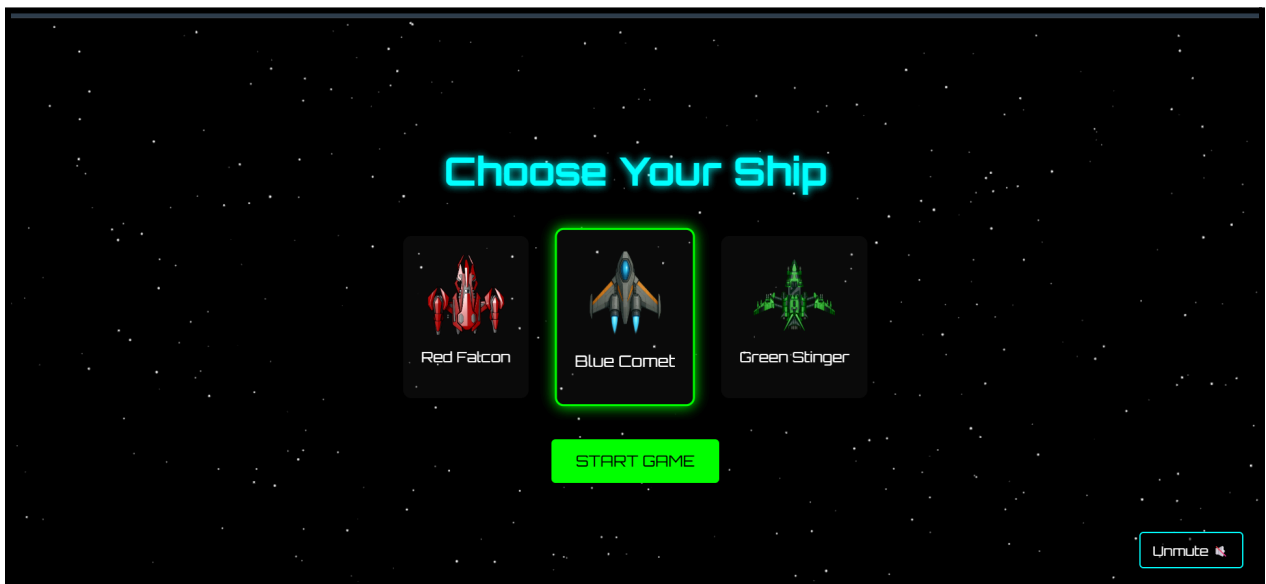


Fig.1.17(Customize Spaceship Page)

5.4.Main Game Page :



Fig.1.18(Main Game)

5.5.Powerup And Bot :

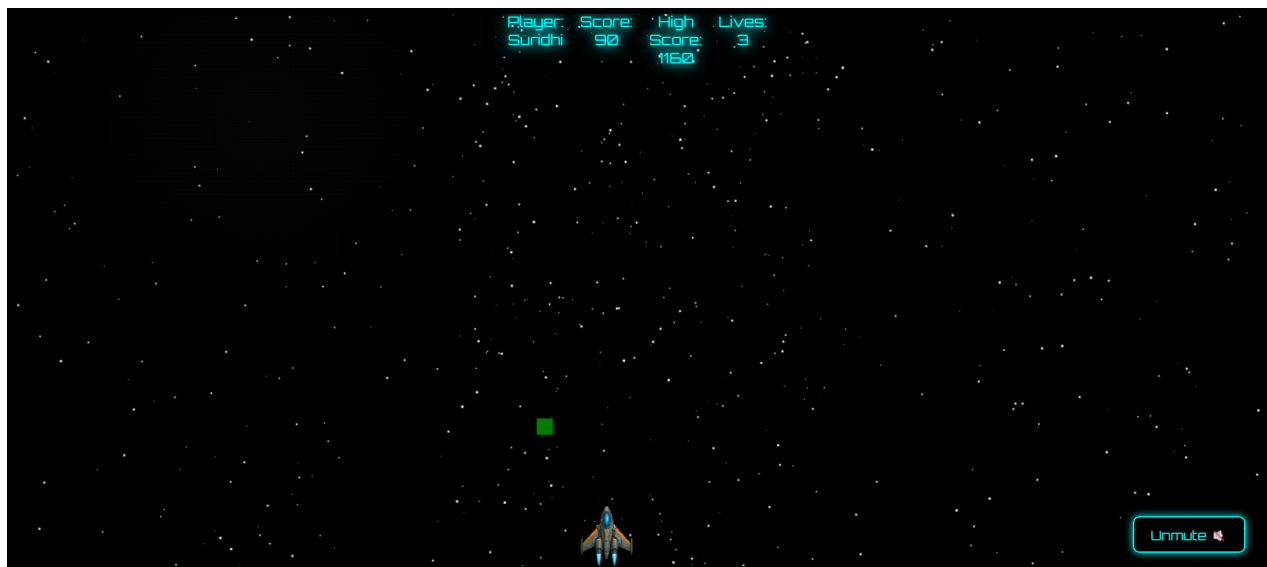


Fig.1.19(Powerup[Lives])



Fig.1.20(Bot)

5.6.End Game Page :

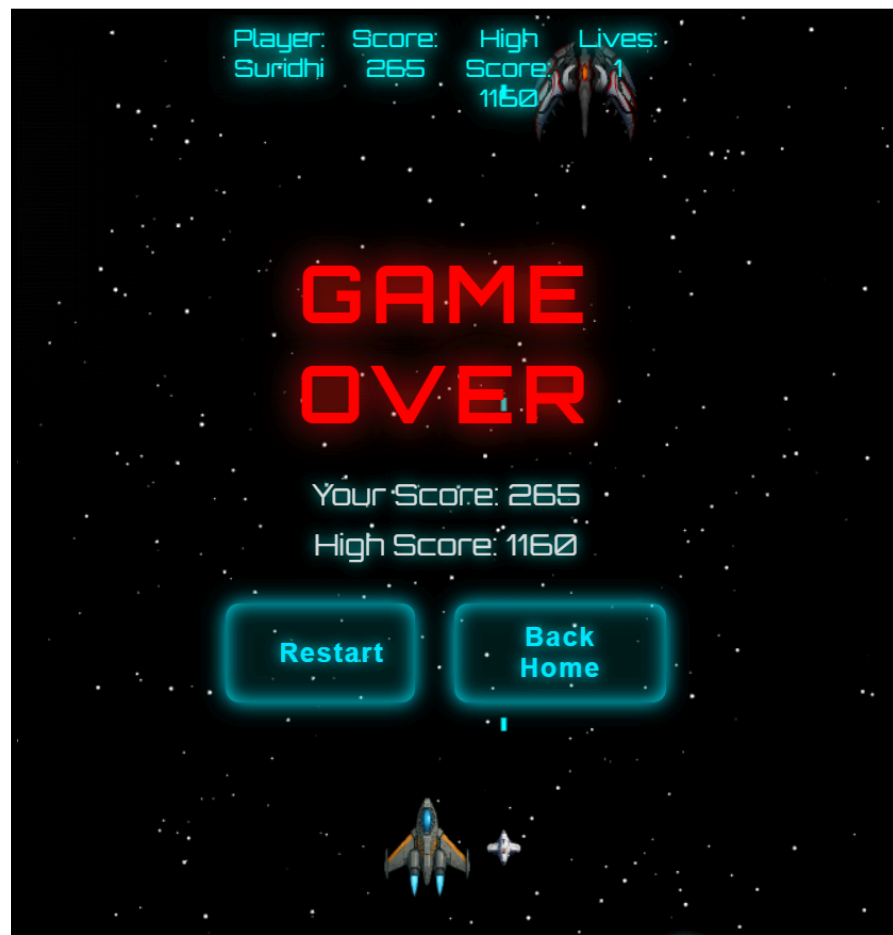


Fig.1.21(End Game)

Chapter 6

CONCLUSION

The Space Shooter Game successfully demonstrates how HTML, CSS, and JavaScript can be used together to create an interactive, visually appealing, and fully functional browser-based application. Through this project, we were able to implement real-time animations, user-controlled gameplay, enemy interactions, collision detection, scoring, and background effects—all running smoothly within a web browser. The modular structure of the code, along with separate pages for the homepage, instructions, customization, and gameplay, helped us understand how to organize a multi-page web project effectively.

Developing this game provided hands-on experience in event handling, DOM manipulation, canvas-based rendering, and responsive UI design. It also strengthened our understanding of game loops, asset management, and sound integration. Overall, the project not only improved our technical skills but also enhanced our creativity and problem-solving abilities. The Space Shooter Game serves as a strong foundation for building more advanced web-based games and interactive applications in the future.

REFERENCES

1. **MDN Web Docs – HTML Basics**
https://developer.mozilla.org/en-US/docs/Learn/Getting_started_with_the_web/HTML_basics
2. **MDN Web Docs – CSS: Cascading Style Sheets**
<https://developer.mozilla.org/en-US/docs/Web/CSS>
3. **MDN Web Docs – JavaScript Guide**
<https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide>
4. **W3Schools – JavaScript Game Development Tutorial**
https://www.w3schools.com/graphics/game_intro.asp